Today, I want to share my experience of participating in one of Kaggle competition, **Quick, Draw! Doodle Recognition Challenge** I will briefly introduce the challenge and start to explain dataset and others. This post may have the following sections.

1. About this competition
2. Dataset
3. Feature extraction
4. Model selection
5. Ensemble
6. Conclusion & Further question

You can also refer to my **github repository** for my source code.

About this competition

In this competition, we are asked to build a model that classifies hand-drawn images. The model you build will classify 340 labels, and your model will be ranked based on top-3 accuracy of your model prediction.

Dataset

Datasets are a set of CSV files. Each CSV file is a collection of images that people drew and the name of CSV file is a target that people are asked to draw. For example, in ant.csv file, all ant images that people draw were collected, including the key_id and country code, i.e. the country code where each image was drawn.

In this competition, participants are asked to classify images each of which are one of 340 labels. The list of labels are [LABEL]. Also it is important to note that each drawn image is provided in raw format and simplified format.

Raw format dataset

Raw format of data means no preprocessing has been applied on this dataset. People drew each target in their own devices, such as his i-phone, i-pad, desktop PC, or anything else. No matter what devices were used, raw-formatted CSV file contains values of each image including the key_id and country code. Figure 1 explains how an image is handled under the raw-format.

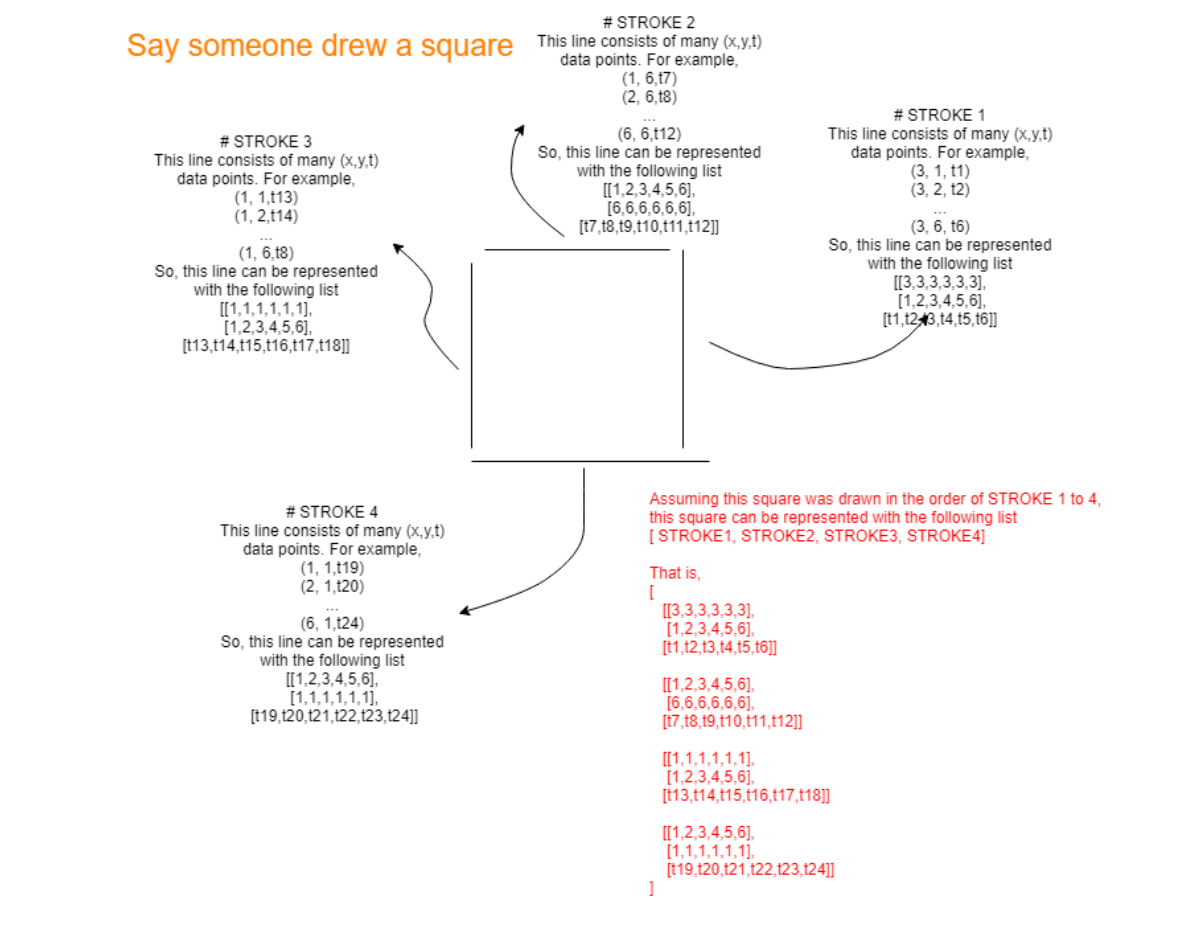

Figure 1: Representation of image data

The raw-formatted train file is provided as a single ZIP file which contains list of following files.

- The Eiffel Tower.csv: Image of Eiffel Tower has been collected.
- The Great Wall of China.csv: Image of Great wall of China has been collected. ...
- ant.csv: Image of ant has been collected. In a raw-formatted ZIP train file, there are 340 CSV files, because we are asked to classify 340 classes.

In this competition, we are supposed to be ranked from the accuracy of test dataset. This test dataset has xxxxx images and provided in raw-format.

Simplified format dataset

Everything is same as raw-formatted dataset, but few difference should be noted.

- Each data point value is ranged from 0 to 255 (somehow normalized)
- Each data point value is an int type.
- No time data is provided for each image. That is, only (x, y) data is available.
- Test dataset is provided in simplified format (NOT raw-file format)

Read Raw/Simplified formatted dataset

Watching is better than Reading. For those who does not have enough time to read, dataset can be read as shown below.

Firstly, we can read the raw-formatted file like this.

```
import pandas as pd
raw = pd.read_csv('/data/doodle/raw/The Eiffel Tower.csv')
raw.head()
```

And we can read the simplified-formatted file like this.

```
import pandas as pd
simp = pd.read_csv('/data/doodle/simplified/The Eiffel Tower.csv')
simp.head()
```

Basically, whether it is raw-formatted or simplified one, the entire column does not change. However the size of drawing column is much smaller with simplified-formatted file due to the following reasons

- simplified image has less data points than raw image.
- simplified image does not have time data.

Feature extraction

Re-organizing and shuffle dataset

Before we extract feature, we should re-organize and shuffle our dataset, because each CSV file has only one label. We are going to read a chunck of data from each CSV and make a new CSV file which contains 340 different labels. Then to save capacity of disk space, we compressed the new CSV files. In this way, we created 100 newly-compressed CSV files all of which contain 340 labels, i.e. well-shuffled. Whole process of re-organization of dataset is explained in Figure 2.

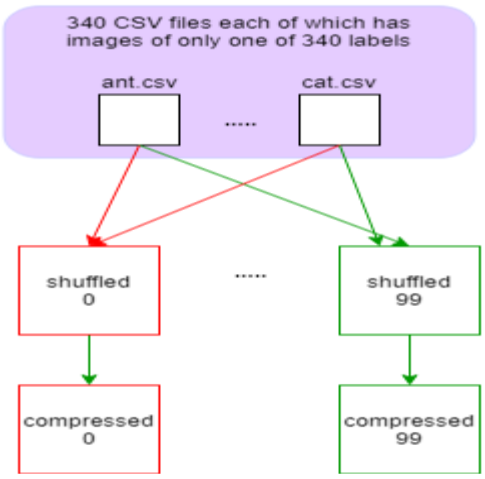

Figure 2: Re-organization and shuffling of original data

Also note that you should run the above process for simplified format and raw format both, if you want to use both of them.

Transforming images into 3 channel.

As explained in Figure 1, image data is just a collection of (x, y). We should convert this form of image into (size, size, channel) shape, like a normal image data. Using CV2 library in Python, we could convert (x,y) dot images into (size, size, channel)-shaped image. Of course, this conversion is not necessary if you don't want to use CNN-like model. However, since we do not want to focus on type of deep learning model for this competition, we just selected CNN model, therefore we had to do this conversion.

Think about what feature may be useful

It is time to think about what kind of features are useful for classifying hand-drawn image. In doodle, since every image is black-and-white colored, we do not need to think about actual R,G,B colors with 3 channel. Instead of RGB, we should consider different features that may be useful to classify images.

Think about drawing an i-phone. you will draw large vertical-long square and then screen and the button at the bottom. Most of people may draw in this order. I want to say that the order of drawing each stroke may be useful one of useful feature. To see if it is really useful feature, let's have a look with real data.

I sampled 64 images of "cell phone"(not specifically i-phone). When they are plotted in python, you can see that strokes are represented from yellow to blue. Let me explain how we have got this representation. I said we are not going to use R,G,B color, but we still need to represent our features in the range of 0-255 for each feature. As we are focusing on our first feature now, we need to represent our first feature in the range of 0-255. And our first feature is "the order of stroke".

The code for our first feature is given below.

```
for t, s in enumerate(strokes):
    f0 = 255 - min(t, 10)*13
```

For example, when someone draw a square on doodle, he may use four strokes. Assuming he draw a square in the order of top -> right -> bottom -> left, each stroke would contain 255, 242, 229, 216. You can see the detail in Figure 3.

```
In [120]: img = np.array([[0,   0,   0,   0,   0,   0,   0,   0,   0,   0],
[0, 255, 255, 255, 255, 255, 255, 255,   0,   0],
[0,   0,   0,   0,   0,   0,   0,   0, 242,   0],
[0, 216,   0,   0,   0,   0,   0,   0, 242,   0],
[0, 216,   0,   0,   0,   0,   0,   0, 242,   0],
[0, 216,   0,   0,   0,   0,   0,   0, 242,   0],
[0, 216,   0,   0,   0,   0,   0,   0, 242,   0],
[0,   0, 229, 229, 229, 229, 229, 229,   0,   0],
[0,   0,   0,   0,   0,   0,   0,   0,   0,   0],
[0,   0,   0,   0,   0,   0,   0,   0,   0,   0]])

In [121]: plt.imshow(img)
print(img.sum())
```

```
5233
```

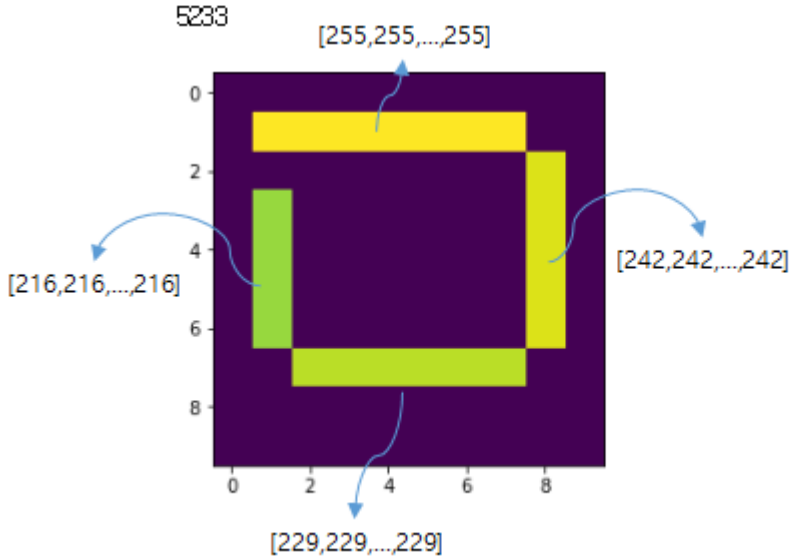

Figure 3: Our first feature: the order of stroke

Our second feature is the number of points in one stroke for an unit time, i.e. how many points someone produce in a given time. This feature can be produced with the following code.

```
for t, s in enumerate(strokes):
    n_points = len(s)  # the number of points in one stroke
    for i in range(n_points - 1):
        st = stroke[2][i]        # time where ith point is produced
        dt = stroke[2][i+1] # time where i+1th point is produced
        time = abs(dt - st)
        time = 1 if time == 0 else time # to avoid divide-by-zero error
        f1 = min(n_points, 255)/np.sqrt(time)
```

Our third feature is the length of point-point divided by time, i.e. velocity. In one stroke, you can see many pairs of points. we measured the distance between those two points and divided the distance into time.

```
for t, s in enumerate(strokes):
    n_points = len(s)  # the number of points in one stroke
    for i in range(n_points - 1):
        t = stroke[2][i] # time where ith point is produced
        dt = stroke[2][i+1] # time where i+1th point is produced
        time = abs(dt - st)
        time = 1 if time == 0 else time # to avoid divide-by-zero error

        # assuming sx, sy are the (x, y) pair of source point
        # and dx, dy are the (x, y) pair of destination point
        f2 = min(int((np.sqrt((sx-dx)*(sx-dx) + (sy-dy)*(sy-dy)) / time)*255.0),
255)
```

By stacking up our first, second, third features, we can make (size, size, 3) shape of data for each image whose values are ranged from 0 to 255. To provide our features into our model, we normalized our features to the range of 0-1.

How to evaluate features

It is an important question. How can we verify our feature is good enough? It is actually hard question. But, for this competition, I decided to evaluate the quality of my features with correlation coefficient. If my feature represent almost same characteristics, then correlation coefficient will be close to one. In this case, we should consider to remove one of them, because those features are almost same features. To calculate the correlation coefficient, we unstacked (128, 128, 3) into three (128, 128) and flattened them. Next we compared f0, f1, f2 by calculating correlation coefficient. You can understand easily with Figure 4 below.

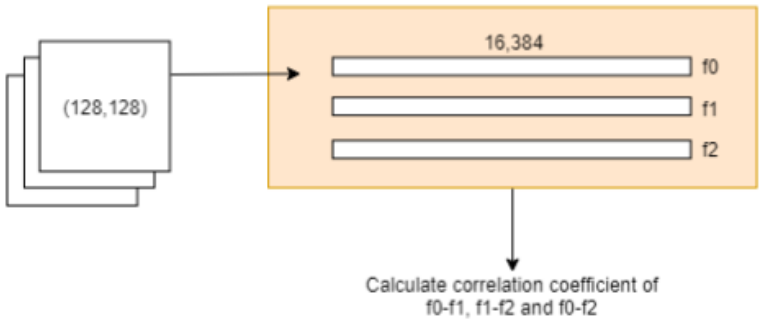

Figure 4: Correlation coefficient of features

In Figure 4, we have got three correlation coefficients, i.e. f0-f1, f1-f2 and f0-f2. Getting correlation coefficients from only one image is not enough. So, we did the same things over and aver with 34,000 images, therefore collected 34,000 set of correlation coefficient for each feature. For each feature we can plot a histogram and the result is Figure 5.

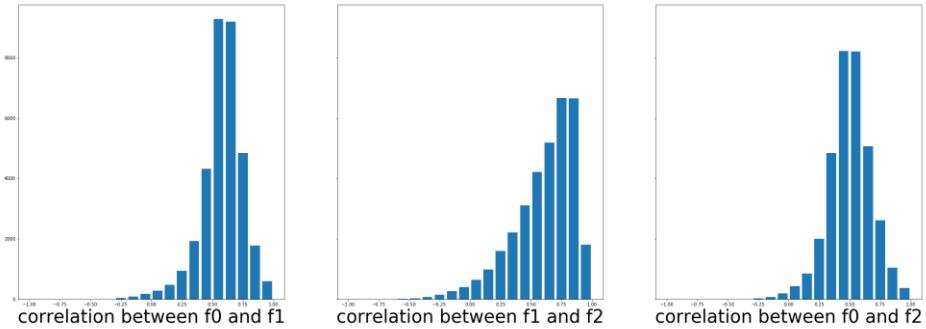

Figure 5: Histogram of correlation coefficient

All features are slightly right-biased. Majority of images have correlation coefficient value from 0.50 to 0.75. For my opinion, I think it is okay to use our three features, since it is not very biased(I know someone might think it is too much. If you think so, please leave me any comment. It will be very helpful)

Model selection

Actually, for this competition, I didn't carefully consider about modelling part. Rather than modelling part, I would like to spend more time for feature selection. Therefore, I just used some pre-implemented models in keras_applications, such as MobileNet, Inception Resnet V2, Xception and so on. I have built four models in total. They are

- Inception Resnet V2
- Inception Resnet V2(imagenet pre-trained)
- Xception
- Xception(imagenet pre-trained)

Here is the parameters of the above models.

- Optimizer: AdamOptimizer
- Learning Rate: 0.002
- Image Size: 128 for Inception Resnet V2, 115 for Xception
- Batch Size: from 400 to 500, according to the memory limit
- Decay learning rate: multipy 0.75 on the old learning rate

Here is the result of each models for public/private Leaderboard(LB).

Models	Use Imagenet	Public LB	Private LB
Inception Resnet	O	0.92898	0.93034

Models	Use Imagenet	Public LB	Private LB
Inception Resnet	X	0.93123	0.92961
Xception	O	0.91699	0.91851
Xception	X	0.93040	0.92964

One thing to note is that I just decided to use the pretrained-Xception model(3rd line) even though its accuracy is far less than others. I wanted to know the effect of ensemble, which you can see in the next chapter.

Ensemble

Even though Pretrained Xception model showed us comparably less accuracy than others, I wanted to know the effect of ensemble. Which result will be better?? Is it okay to have or not? The answer is that it is good to have. When we ensemble those models into one, we were able to reach accuracy of 0.94034.

Further question & Conclusion

I have several questions on this competition.

- I have used correlation coefficient to evaluate my features. However, I don't think it is official or recommanded way to do it. Any other suggestion?
- How to use better ensemble learning such as XgBoost or LightGBM? If I do want to use Xgboost or LightGBM, should I build my model from the scratch again?? Or it is just post-processing?

Actually, this is my first time to fully participate in the Kaggle competition. I know I could not follow the official timeline. But that is not the important point. Learning from the challenge is important. Although I could not make up fully official rank at this time, someday... who knows :)

This result was obtained by mean average ensembling. That is, add probability of all 340 labels and divided by the number of model, i.e. 4.